



საქართველოს ტურისტული გზამკვლევის მობილური აპლიკაცია

გიორგი მანჯავიძე

ინფორმაციული ტექნოლოგიების საბაკალავრო პროგრამა

წარდგენილია აკადემიური ხარისხის მოსაპოვებლად:

ინფორმაციული ტექნოლოგიების ბაკალავრი

ნაშრომის ხელმძღვანელი: ირაკლი კაპანაძე

ბიზნესისა და ტექნოლოგიების უნივერსიტეტი

თბილისი, 2026

განაცხადი

როგორც ავტორი, ვაცხადებ, რომ ნაშრომი წარმოადგენს ჩემს ორიგინალურ ნამუშევარს, ხოლო სხვა ავტორების მიერ შექმნილი მასალები მოხსენებული ან ციტირებულია სათანადო წესების შესაბამისად.

სტუდენტის სახელი და გვარი: გიორგი მანჯავიძე

თარიღი: 08.04.2026

ხელმოწერა:

აბსტრაქტი

საქართველო ტურისტული მიმართულებით სწრაფად მზარდი ქვეყანაა, თუმცა მოგზაურისთვის საჭირო ინფორმაცია გაფანტულია ათეულობით დამოუკიდებელ ვებგვერდზე, სოციალურ ქსელსა და ბეჭდურ გზამკვლევში. ერთიანი, სტრუქტურირებული და მობილურ მოწყობილობაზე მორგებული ციფრული პლატფორმის არარსებობა ართულებს მოგზაურობის დაგეგმვას, განსაკუთრებით რეგიონული ქალაქების მიმართულებით.

ნაშრომის მიზანია საქართველოს ციფრული ტურისტული გზამკვლევის მობილური აპლიკაციის დაპროექტება და რეალიზაცია. შექმნილი პროდუქტი მომხმარებელს საშუალებას აძლევს დაათვალიეროს და მოძებნოს საქართველოს ქალაქები, გაეცნოს მათ დეტალურ აღწერას, ღირსშესანიშნაობების, რესტორნების, კაფეებისა და სასტუმროების ჩამონათვალს, დაათვალიეროს რეკომენდებული ტურისტული მარშრუტები, იხილოს ლოკაციები რუკაზე და შეინახოს რჩეული ქალაქები და ადგილები პირად პროფილში.

სისტემა აგებულია კლიენტ-სერვერულ მოდელზე. მობილური კლიენტი შექმნილია Flutter-ისა და Dart-ის საშუალებით, სუფთა არქიტექტურის (Clean Architecture) პრინციპებისა და Riverpod-ის გამოყენებით. სერვერული ნაწილი აგებულია მიკროსერვისული არქიტექტურით: ავთენტიფიკაციის სერვისი რეალიზებულია ASP.NET Core-ზე, ხოლო ქალაქების, ლოკაციების, მარშრუტებისა და ფავორიტების სერვისები კი Go-ზეა. კლიენტსა და სერვერს შორის კომუნიკაცია მიმდინარეობს REST API-ის მეშვეობით API Gateway-ის გავლით, მიკროსერვისებს შორის კი gRPC პროტოკოლით. მონაცემები ინახება PostgreSQL მონაცემთა ბაზაში, წვდომის კონტროლი უზრუნველყოფილია JWT ტოკენებით.

შედეგად შეიქმნა ფუნქციურად სრული და ტესტირებული პროტოტიპი, რომელიც ადასტურებს არჩეული არქიტექტურული მიდგომის სიცოცხლისუნარიანობას და ქმნის საფუძველს სისტემის შემდგომი განვითარებისთვის.

საკვანძო სიტყვები: მობილური აპლიკაცია, ტურიზმი, Flutter, მიკროსერვისები, gRPC, REST API, PostgreSQL, JWT.

Abstract

Georgia is a rapidly growing tourist destination, yet the information a traveller needs (covering cities, landmarks, routes, dining venues and accommodation) remains scattered across dozens of independent websites, social media pages and printed guidebooks. The absence of a single, structured and mobile-oriented digital platform complicates trip planning, particularly for regional destinations outside the main tourist corridors.

The aim of this thesis is to design and implement a mobile application that serves as a digital tourist guide to Georgia. The resulting product allows users to browse and search Georgian cities, view detailed descriptions, explore lists of attractions, restaurants, cafés and hotels, review recommended tourist routes, display locations on an interactive map, and save favourite cities and places to a personal profile.

The system follows a client–server model. The mobile client is built with Flutter and Dart, applying Clean Architecture principles and Riverpod for state management. The backend is implemented as a set of microservices: the authentication service is written in ASP.NET Core, while the city, places, route and favourite services are written in Go. The client communicates with the backend over a REST API exposed through an API Gateway, while inter-service communication uses the gRPC protocol. Data is persisted in a PostgreSQL database, and access control is enforced through JWT tokens.

The outcome is a functionally complete and tested prototype that demonstrates the viability of the selected architectural approach and provides a foundation for further development of the platform.

Keywords: mobile application, tourism, Flutter, microservices, gRPC, REST API, PostgreSQL, JWT.

სარჩევი

განაცხადი.....	2
----------------	---

აბსტრაქტი	3
Abstract.....	4
სარჩევი	4
1. შესავალი	7
1.1. თემის აქტუალობა.....	7
1.2. არსებული პრობლემა	7
1.3. პროექტის მიზანი	8
1.4. ძირითადი ამოცანები	9
1.5. პროექტის პრაქტიკული მნიშვნელობა	10
1.6. კვლევის მეთოდოლოგია და საკვლევი კითხვები	10
2. თეორიული მიმოხილვა	12
2.1. პროექტთან დაკავშირებული ძირითადი ცნებები	12
2.2. გამოყენებული ტექნოლოგიები.....	13
2.3. მსგავსი სისტემები	15
2.4. შერჩეული მიდგომის უპირატესობა	15
3. სისტემის მოთხოვნები და დიზაინი.....	17
3.1. ფუნქციური მოთხოვნები.....	17
3.2. არაფუნქციური მოთხოვნები	17
3.3. მომხმარებლის როლები.....	18
3.4. სისტემის არქიტექტურა	19
3.5. მონაცემთა ბაზის სტრუქტურა	21
3.6. გამოყენებული ტექნოლოგიური სტეკი	22
4. პრაქტიკული რეალიზაცია.....	24
4.1. სისტემის ძირითადი მოდულები.....	24
4.2. პროგრამის მუშაობის პროცესი.....	25

4.4. მნიშვნელოვანი ტექნიკური გადაწყვეტილებები.....	26
4.4. გაშვების სრული ინსტრუქცია.....	28
5. ტესტირება და შედეგები	29
5.1. ტესტირების მეთოდოლოგია.....	29
5.2. შემოწმებული ფუნქციები და ტესტების შედეგები.....	29
5.3. მიღებული შედეგები	30
5.4. გამოვლენილი ხარვეზები და მათი გამოსწორება.....	30
6. დასკვნა და რეკომენდაციები	32
7. გამოყენებული ლიტერატურა.....	34

1. შესავალი

1.1. თემის აქტუალურობა

ტურიზმი საქართველოს ეკონომიკის ერთ-ერთი წამყვანი დარგია და ქვეყნის მთლიან შიდა პროდუქტში მნიშვნელოვან წილს იკავებს. ბოლო ათწლეულის განმავლობაში საერთაშორისო ვიზიტორთა რიცხვი სტაბილურად იზრდება, პარალელურად კი ვითარდება შიდა ტურიზმიც: ადგილობრივი მოსახლეობა სულ უფრო ხშირად გეგმავს მოკლევადიან მოგზაურობებს ქვეყნის შიგნით. ამ ფონზე იზრდება მოთხოვნა სანდო, სტრუქტურირებულ და სწრაფად ხელმისაწვდომ ტურისტულ ინფორმაციაზე.

პარალელურად, მოგზაურობის დაგეგმვის ქცევითი მოდელი ფუნდამენტურად შეიცვალა. თანამედროვე ტურისტი მარშრუტს აღარ გეგმავს მხოლოდ გამგზავრებამდე; გადაწყვეტილებების მნიშვნელოვან ნაწილს იღებს უკვე ადგილზე, სმარტფონის მეშვეობით: სად ისადილოს, რომელი ღირსშესანიშნაობა ნახოს პირველად, რამდენად შორს არის მეზობელი ქალაქი. სმარტფონი ამგვარად იქცა მოგზაურობის ძირითად ინსტრუმენტად, მობილური აპლიკაცია კი ტურისტულ ინფორმაციასთან წვდომის მთავარ არხად.

სწორედ ამ ორი ტენდენციის (ტურისტული ნაკადის ზრდისა და მოგზაურობის დაგეგმვის მობილიზაციის) გადაკვეთაზე ჩნდება პრაქტიკული საჭიროება: შეიქმნას ერთიანი ციფრული პლატფორმა, რომელიც საქართველოს ტურისტულ შეთავაზებას ორგანიზებულად, ვიზუალურად და მობილურ მოწყობილობაზე მორგებული ფორმით წარმოაჩენს. ამავდროულად, ასეთი სისტემის შექმნა საშუალებას იძლევა პრაქტიკულად იქნას გამოყენებული პროგრამული ინჟინერიის თანამედროვე მიდგომები: მიკროსერვისული არქიტექტურა, კროსპლატფორმული მობილური განვითარება და სერვისებს შორის ეფექტიანი კომუნიკაციის პროტოკოლები, რაც თემას როგორც პრაქტიკულ, ისე აკადემიურ აქტუალობას სძენს.

1.2. არსებული პრობლემა

მიუხედავად ტურისტული რესურსების სიმრავლისა, დღეს მოგზაური რამდენიმე სისტემურ სირთულეს აწყდება:

- ინფორმაციის ფრაგმენტულობა. ქალაქების აღწერა, ღირსშესანიშნაობების ჩამონათვალი, სასტუმროებისა და რესტორნების მონაცემები განთავსებულია ერთმანეთისგან დამოუკიდებელ წყაროებზე. ერთი მოგზაურობის დასაგეგმად მომხმარებელს უწევს რამდენიმე ვებგვერდის, სოციალური ქსელისა და რუკის აპლიკაციის პარალელურად გამოყენება.
- მონაცემთა არაერთგვაროვნება და მოძველება. სხვადასხვა წყაროში ერთი და იმავე ობიექტის შესახებ განსხვავებული ან მოძველებული ინფორმაციაა განთავსებული, რაც ამცირებს ნდობას და ზრდის დაგეგმვის რისკს.
- რეგიონული ქალაქების არასაკმარისი წარმოჩენა. ციფრული რესურსების უმეტესობა ორიენტირებულია რამდენიმე ყველაზე პოპულარულ მიმართულებაზე, ხოლო მცირე ქალაქები, რომლებსაც მნიშვნელოვანი ტურისტული პოტენციალი აქვთ, პრაქტიკულად უხილავი რჩება.
- გლობალური პლატფორმების ლოკალიზაციის დაბალი დონე. საერთაშორისო სერვისები საქართველოს მხოლოდ ერთ-ერთ მიმართულებად განიხილავენ; მათში წარმოდგენილი კონტენტი ხშირად არასრულია, ქართულენოვანი მხარდაჭერა შეზღუდულია, ხოლო ლოკალური კონტექსტი (ტრადიციები, სეზონურობა, რეგიონული სამზარეულო) გათვალისწინებული არ არის.
- პერსონალიზაციის ნაკლებობა. მომხმარებელს არ აქვს ერთიანი სივრცე, სადაც შეინახავს მისთვის საინტერესო ქალაქებსა და ლოკაციებს და დაუბრუნდება მათ მოგზაურობის დროს.

ჩამოთვლილი პრობლემები ერთობლიობაში ქმნის ბარიერს როგორც ტურისტისთვის, ისე ტურისტული ობიექტებისთვის, რომლებიც პოტენციურ ვიზიტორამდე ვერ აღწევენ.

1.3. პროექტის მიზანი

პროექტის მიზანია საქართველოს ტურისტული გზამკვლევის მობილური აპლიკაციის დაპროექტება და პრაქტიკული რეალიზაცია: ერთიანი პლატფორმის შექმნა, რომელიც ტურისტსა და ადგილობრივ მომხმარებელს სტრუქტურირებული სახით მიაწვდის ინფორმაციას საქართველოს ქალაქების, ღირსშესანიშნაობების, ტურისტული მარშრუტების, კვების ობიექტებისა და განთავსების საშუალებების

შესახებ, გაამარტივებს მოგზაურობის დაგეგმვას და ხელს შეუწყობს ახალი ადგილების აღმოჩენას.

მიზნის მეორე, ტექნიკური განზომილებაა თანამედროვე პროგრამული უზრუნველყოფის არქიტექტურის (მიკროსერვისული მიდგომის, სუფთა არქიტექტურისა და სერვისებს შორის კომუნიკაციის თანამედროვე პროტოკოლების) პრაქტიკული გამოყენების დემონსტრირება რეალურ, დასრულებულ პროდუქტზე.

1.4. ძირითადი ამოცანები

დასახული მიზნის მისაღწევად განისაზღვრა შემდეგი ამოცანები:

1. არსებული ტურისტული ციფრული პლატფორმების ანალიზი და მათი ძლიერი და სუსტი მხარეების გამოვლენა.
2. სისტემის ფუნქციური და არაფუნქციური მოთხოვნების ჩამოყალიბება.
3. მიკროსერვისებზე დაფუძნებული სისტემის არქიტექტურის დაპროექტება და სერვისებს შორის პასუხისმგებლობების გამიჯვნა.
4. მონაცემთა ბაზის ლოგიკური და ფიზიკური სტრუქტურის შემუშავება.
5. ავთენტიფიკაციისა და ავტორიზაციის მექანიზმის რეალიზაცია JWT ტოკენების საფუძველზე.
6. სერვერული მიკროსერვისების რეალიზაცია Go და ASP.NET Core ტექნოლოგიებზე, API Gateway-ის მეშვეობით ერთიანი REST ინტერფეისის მიწოდებით.
7. Flutter-ზე დაფუძნებული მობილური კლიენტის დაპროექტება და რეალიზაცია სუფთა არქიტექტურის პრინციპებით.
8. სისტემის ტესტირება: ერთეულოვანი, ინტეგრაციული და ფუნქციური ტესტების ჩატარება, გამოვლენილი ხარვეზების აღმოფხვრა.
9. მიღებული შედეგების შეფასება და სისტემის შემდგომი განვითარების რეკომენდაციების ჩამოყალიბება.

1.5. პროექტის პრაქტიკული მნიშვნელობა

პროექტის პრაქტიკული ღირებულება რამდენიმე დონეზე ვლინდება. მომხმარებლისთვის ის ამცირებს მოგზაურობის დაგეგმვაზე დახარჯულ დროს და ერთ ინტერფეისში აერთიანებს იმ ინფორმაციას, რომლის მოსაძიებლადაც დღეს რამდენიმე დამოუკიდებელი რესურსია საჭირო. ტურისტული ობიექტებისთვის (სასტუმროებისთვის, რესტორნებისა და მუზეუმებისთვის) პლატფორმა წარმოადგენს დამატებით არხს პოტენციურ ვიზიტორამდე მისასვლელად, განსაკუთრებით რეგიონებში.

ტექნიკური თვალსაზრისით, პროექტი წარმოადგენს სრულ, დოკუმენტირებულ მაგალითს იმისა, თუ როგორ შეიძლება ერთ სისტემაში თანაარსებობდნენ სხვადასხვა ტექნოლოგიურ სტეკზე დაწერილი სერვისები. სისტემის მოდულური აგებულება საშუალებას იძლევა მომავალში დაემატოს ახალი ფუნქციონალი (მაგალითად, მომხმარებელთა შეფასებები, ონლაინ ჯავშნის ინტეგრაცია ან ოფლაინ რეჟიმი) არსებული კომპონენტების გადაწერის გარეშე.

1.6. კვლევის მეთოდოლოგია და საკვლევი კითხვები

ნაშრომი ეყრდნობა გამოყენებით, საინჟინრო კვლევის მიდგომას, რომელიც ცნობილია როგორც დაპროექტებასა და რეალიზაციაზე დაფუძნებული კვლევა (design and development research). ამ მიდგომის ფარგლებში ცოდნა მიიღება რეალური პროგრამული არტეფაქტის, ანუ ტურისტული გზამკვლევის აპლიკაციის, დაპროექტების, აგებისა და შეფასების გზით.

კვლევა რამდენიმე თანმიმდევრულ ეტაპად წარიმართა:

- პრობლემის იდენტიფიცირება და არსებული, მსგავსი ტურისტული პლატფორმების ანალიზი;
- სისტემის ფუნქციური და არაფუნქციური მოთხოვნების ჩამოყალიბება;
- არქიტექტურისა და მონაცემთა ბაზის დაპროექტება;
- სისტემის იტერაციული რეალიზაცია მობილურ კლიენტად და სერვერულ მიკროსერვისებად;

- აგებული სისტემის ტესტირება და მიღებული შედეგების შეფასება დასახულ მოთხოვნებთან მიმართებით.

შეფასების ეტაპზე გამოყენებულია შავი და თეთრი ყუთის ტესტირების მეთოდები, API-ის ხელით ტესტირება Postman-ის საშუალებით, აგრეთვე მიმღებლობითი ტესტირება მომხმარებელთა მცირე ჯგუფის მონაწილეობით. მიღებული შედეგები დეტალურად განხილულია მე-5 თავში.

ზემოაღნიშნული მიდგომა მიმართულია შემდეგ საკვლევ კითხვებზე პასუხის გასაცემად:

1. რამდენად შესაძლებელია საქართველოს ტურისტული ინფორმაციის (ქალაქები, ღირსშესანიშნაობები, მარშრუტები, კვებისა და განთავსების ობიექტები) ერთიან, სტრუქტურირებულ მობილურ პლატფორმად გაერთიანება?
2. რა უპირატესობებსა და ხარჯებს წარმოშობს მიკროსერვისული არქიტექტურის გამოყენება მონოლითურ მიდგომასთან შედარებით ამ ტიპის სისტემისთვის?
3. როგორ აისახება სერვისებს შორის gRPC პროტოკოლის გამოყენება სისტემის წარმადობასა და პასუხის დროზე REST-თან შედარებით?
4. აკმაყოფილებს თუ არა შექმნილი პროტოტიპი დასახულ ფუნქციურ და არაფუნქციურ მოთხოვნებს ტესტირების შედეგების მიხედვით?

2. თეორიული მიმოხილვა

2.1. პროექტთან დაკავშირებული ძირითადი ცნებები

მობილური აპლიკაცია არის პროგრამული უზრუნველყოფა, რომელიც შექმნილია სმარტფონსა და პლანშეტზე გასაშვებად. მისი რეალიზაციის სამი ძირითადი მიდგომა არსებობს: ნატიური (თითოეული პლატფორმისთვის ცალკე კოდი), ჰიბრიდული (ვებ-ტექნოლოგიები კონტეინერში) და კროსპლატფორმული, სადაც ერთი კოდის ბაზა კომპილირდება რამდენიმე პლატფორმისთვის. წინამდებარე პროექტში არჩეულია სწორედ კროსპლატფორმული მიდგომა.

კლიენტ-სერვერული არქიტექტურა გულისხმობს სისტემის დაყოფას მოთხოვნის ინიციატორ კლიენტად და მოთხოვნის დამმუშავებელ სერვერად. მობილური აპლიკაცია ამ მოდელში კლიენტის როლს ასრულებს, ბიზნეს-ლოგიკისა და მონაცემთა შენახვის პასუხისმგებლობა კი სერვერულ ნაწილზეა გადატანილი.

მონოლითური და მიკროსერვისული არქიტექტურა. მონოლითურ სისტემაში მთელი ფუნქციონალი ერთ, განუყოფელ აპლიკაციად ვითარდება და იშლება. მიკროსერვისული მიდგომა, პირიქით, სისტემას ყოფს მცირე, დამოუკიდებლად გაშვებად სერვისებად, რომელთაგან თითოეული ერთ ბიზნეს-შესაძლებლობაზეა პასუხისმგებელი და საკუთარ მონაცემებს ფლობს (Newman 2021). ეს მიდგომა ზრდის სისტემის მასშტაბირებადობასა და მდგრადობას, თუმცა ამატებს ოპერაციულ სირთულეს.

API (Application Programming Interface) არის კონტრაქტი, რომელიც განსაზღვრავს, თუ როგორ უნდა დაუკავშირდეს ერთი პროგრამული კომპონენტი მეორეს: რა მოთხოვნის გაგზავნა შეიძლება, რა ფორმატით და რა პასუხის მოლოდინია.

ავთენტიფიკაცია და ავტორიზაცია. ავთენტიფიკაცია პასუხობს კითხვას „ვინ არის მომხმარებელი“, ავტორიზაცია კი პასუხობს კითხვას „რის გაკეთების უფლება აქვს მას“. სისტემაში ორივე პროცესი ერთმანეთისგან მკაფიოდაა გამიჯნული.

მდგომარეობის მართვა (State Management) მობილურ განვითარებაში აღნიშნავს იმ მექანიზმს, რომლითაც აპლიკაცია ინახავს, ცვლის და ინტერფეისზე ასახავს

მიმდინარე მონაცემებს, მაგალითად, ჩატვირთულ ქალაქთა სიას ან ავტორიზებული მომხმარებლის სტატუსს.

2.2. გამოყენებული ტექნოლოგიები

სისტემის რეალიზაციისთვის შერჩეული ტექნოლოგიები პირობითად სამ ჯგუფად იყოფა: მობილური კლიენტი, სერვერული ნაწილი და მონაცემთა შენახვის ფენა.

მობილური კლიენტი

- Flutter არის Google-ის ღია კოდის მქონე კროსპლატფორმული ფრეიმვორკი, რომელიც ერთი კოდის ბაზიდან საშუალებას იძლევა აიგოს ნატიურთან მიახლოებული წარმადობის აპლიკაცია Android-ისა და iOS-ისთვის. Flutter არ იყენებს პლატფორმის სტანდარტულ ინტერფეისის ელემენტებს, არამედ თავად ხატავს მათ საკუთარი გრაფიკული ძრავით, რაც უზრუნველყოფს ვიზუალურ იდენტურობას პლატფორმებს შორის.
- Dart წარმოადგენს Flutter-ის საბაზისო პროგრამირების ენას. მისი მნიშვნელოვანი უპირატესობაა ორმაგი კომპილაციის მოდელი: განვითარების პროცესში JIT-კომპილაცია უზრუნველყოფს ცვლილებების მყისიერ ასახვას (Hot Reload), საბოლოო აწყობისას კი AOT-კომპილაცია ქმნის ნატიურ მანქანურ კოდს.
- Clean Architecture არის მიდგომა, რომელიც აპლიკაციას ჰყოფს კონცენტრულ ფენებად და ადგენს დამოკიდებულების წესს: შიდა ფენა არასდროს არის დამოკიდებული გარე ფენაზე (Martin 2017). პროექტში გამოყოფილია სამი ფენა: Presentation, Domain და Data.
- Riverpod წარმოადგენს Flutter-ის მდგომარეობის მართვის ბიბლიოთეკას, რომელიც კომპილაციის ეტაპზევე უზრუნველყოფს დამოკიდებულებათა უსაფრთხოებას და, წინა თაობის გადაწყვეტებისგან განსხვავებით, არ არის მიზმული ვიჯეტების ხეზე. ეს აადვილებს ბიზნეს-ლოგიკის ტესტირებას ინტერფეისისგან დამოუკიდებლად.

სერვერული ნაწილი

- Go (Golang) არის სტატიკურად ტიპიზებული, კომპილირებადი ენა, რომელიც შემუშავდა სწორედ ქსელური სერვისების ეფექტიანი რეალიზაციისთვის. მისი მსუბუქი კონკურენტული მოდელი (goroutine-ები და არხები) საშუალებას იძლევა ერთდროულად ათასობით მოთხოვნა დამუშავდეს მცირე რესურსული დანახარჯით (Donovan and Kernighan 2016). გარდა ამისა, Go აწარმოებს ერთ სტატიკურ ბინარულ ფაილს, რაც კონტეინერიზაციას მნიშვნელოვნად ამარტივებს.
- ASP.NET Core წარმოადგენს Microsoft-ის კროსპლატფორმულ ვებ-ფრეიმვორკს, რომელიც მდიდარ, მზა ინფრასტრუქტურას სთავაზობს ავთენტიფიკაციისა და უსაფრთხოების ამოცანებისთვის: პაროლის ჰეშირება, ტოკენების გენერაცია და ვალიდაცია, მომხმარებელთა მართვა.
- REST API არის არქიტექტურული სტილი, რომელიც როი ფილდინგმა ჩამოაყალიბა და რომელიც რესურსზე ორიენტირებულ, უმდგომარეო კომუნიკაციას ეფუძნება HTTP პროტოკოლის სტანდარტული მეთოდების გამოყენებით (Fielding 2000). REST არჩეულ იქნა კლიენტსა და სერვერს შორის კომუნიკაციისთვის სიმარტივისა და ფართო მხარდაჭერის გამო.
- gRPC არის Google-ის მიერ შემუშავებული დისტანციური პროცედურის გამოძახების ჩარჩო, რომელიც მუშაობს HTTP/2-ზე და მონაცემებს გადასცემს Protocol Buffers ბინარულ ფორმატში. ტექსტურ JSON-თან შედარებით ბინარული სერიალიზაცია მნიშვნელოვნად ამცირებს გადაცემული მონაცემების მოცულობასა და დამუშავების დროს, რაც სერვისებს შორის შიდა კომუნიკაციისთვის ოპტიმალურ არჩევანს ქმნის.
- JWT (JSON Web Token) არის ღია სტანდარტი, რომელიც განსაზღვრავს კომპაქტურ, ციფრულად ხელმოწერილ ტოკენს მხარეებს შორის მოთხოვნების უსაფრთხოდ გადასაცემად (Jones, Bradley, and Sakimura 2015). ტოკენი შედგება სამი ნაწილისგან (სათაური, დატვირთვა და ხელმოწერა) და საშუალებას აძლევს სერვერს, დაადასტუროს მომხმარებლის ვინაობა სესიის მდგომარეობის შენახვის გარეშე.

მონაცემთა შენახვა

- PostgreSQL არის ღია კოდის ობიექტ-რელაციური მონაცემთა ბაზის მართვის სისტემა, რომელიც სრულად აკმაყოფილებს ACID პრინციპებს და მხარს უჭერს რთულ ტიპებს, ინდექსების მრავალფეროვან სახეობებსა და გეოგრაფიულ მონაცემებთან მუშაობას PostGIS გაფართოების მეშვეობით, რაც ლოკაციებზე ორიენტირებული სისტემისთვის განსაკუთრებით მნიშვნელოვანია.

2.3. მსგავსი სისტემები

პროექტზე მუშაობის საწყის ეტაპზე გაანალიზდა რამდენიმე არსებული პლატფორმა, რომლებიც ტურისტულ ინფორმაციას გვთავაზობენ. ანალიზის მიზანი იყო მათი ძლიერი მხარეების იდენტიფიცირება და იმ ხარვეზების გამოვლენა, რომელთა შევსებაც შემუშავებულ სისტემას შეუძლია.

სისტემა	ძირითადი დანიშნულება	შეზღუდვები საქართველოს კონტექსტში
TripAdvisor	მომხმარებელთა შეფასებებზე დაფუძნებული გლობალური კატალოგი	კონტენტი კონცენტრირებულია მსხვილ ქალაქებზე; ქართულენოვანი მხარდაჭერა შეზღუდულია; მარშრუტების დაგეგმვის ფუნქცია სუსტია
Google Maps	ნავიგაცია და ობიექტების ძებნა რუკაზე	არ არის ტურისტულ სცენარზე მორგებული; არ სთავაზობს კურიერულ მარშრუტებსა და თემატურ აღწერებს
Wanderlog / Komoot	მოგზაურობის დაგეგმვა და მარშრუტების აგება	ორიენტირებულია დასავლეთ ევროპისა და ჩრდილოეთ ამერიკის ბაზარზე; საქართველოს შესახებ მონაცემები ფრაგმენტულია
georgia.travel	ეროვნული ტურიზმის ოფიციალური საინფორმაციო პორტალი	ძირითადად ვებზე ორიენტირებული; არ სთავაზობს პერსონალიზაციასა და ფავორიტების შენახვას
ლოკალური ბლოგები და სოციალური ქსელები	ავტორისეული რეკომენდაციები და მიმოხილვები	ინფორმაცია არასტრუქტურირებულია, არ არის ძებნადი და სწრაფად ძველდება

2.4. შერჩეული მიდგომის უპირატესობა

ჩატარებული ანალიზის საფუძველზე ჩამოყალიბდა შემუშავებული სისტემის განმასხვავებელი მახასიათებლები. პირველი და მთავარია ვიწრო გეოგრაფიული ფოკუსი: სისტემა მთლიანად საქართველოზეა ორიენტირებული, რაც საშუალებას იძლევა კონტენტი გაცილებით სიღრმისეული და ლოკალურ კონტექსტზე მორგებული იყოს, ვიდრე გლობალურ პლატფორმებზე.

მეორეა ინფორმაციის ერთიანობა: ქალაქი, მისი ღირსშესანიშნაობები, კვების ობიექტები, სასტუმროები და მარშრუტები ერთ ლოგიკურ ერთეულადაა დაკავშირებული, რაც მომხმარებელს ერთ ეკრანზე აძლევს დაგეგმვისთვის საჭირო სრულ სურათს.

ტექნიკური თვალსაზრისით, არჩეული არქიტექტურული მიდგომა შემდეგ უპირატესობებს იძლევა:

- დამოუკიდებელი მასშტაბირება: თუ ლოკაციების სერვისზე დატვირთვა გაიზრდება, შესაძლებელია მხოლოდ მისი ინსტანციების გაზრდა, დანარჩენი სისტემის შეხების გარეშე.
- ტექნოლოგიური მოქნილობა: თითოეული სერვისისთვის შესაძლებელია იმ ენის შერჩევა, რომელიც კონკრეტულ ამოცანას საუკეთესოდ ერგება; სწორედ ამიტომ ავთენტიფიკაცია რეალიზებულია ASP.NET Core-ზე, ხოლო მაღალი დატვირთვის მქონე საკითხავი სერვისები კი Go-ზეა.
- მდგრადობა: ერთი სერვისის მწყობრიდან გამოსვლა არ იწვევს მთელი სისტემის გაჩერებას; მაგალითად, ფავორიტების სერვისის დროებითი მიუწვდომლობის შემთხვევაშიც მომხმარებელი ინარჩუნებს ქალაქებისა და ლოკაციების დათვალიერების შესაძლებლობას.
- შიდა კომუნიკაციის ეფექტიანობა: gRPC-ის ბინარული ფორმატი ამცირებს სერვისებს შორის გადაცემული მონაცემების მოცულობას, რაც პასუხის საერთო დროზე პირდაპირ აისახება.
- კოდის ტესტირებადობა: სუფთა არქიტექტურის ფენობრივი გამიჯვნის წყალობით ბიზნეს-ლოგიკა ინტერფეისისა და ქსელური ფენისგან დამოუკიდებლად ტესტირდება.

3. სისტემის მოთხოვნები და დიზაინი

3.1. ფუნქციური მოთხოვნები

ფუნქციური მოთხოვნები აღწერს, თუ კონკრეტულად რა უნდა შეასრულოს სისტემამ მომხმარებლის თვალსაზრისით. ქვემოთ მოცემულ ცხრილში მოთხოვნები დაჯგუფებულია ფუნქციური მიმართულებებით და პრიორიტეტის მიხედვით.

კოდი	მოთხოვნა	პრიორიტეტი
FR-01	მომხმარებელს უნდა შეეძლოს ელფოსტითა და პაროლით რეგისტრაცია	მაღალი
FR-02	მომხმარებელს უნდა შეეძლოს სისტემაში ავტორიზაცია და გამოსვლა	მაღალი
FR-03	სისტემამ უნდა უზრუნველყოს JWT ტოკენის გაცემა და მისი განახლება	მაღალი
FR-04	მომხმარებელს უნდა შეეძლოს საქართველოს ქალაქების სიის დათვალიერება	მაღალი
FR-05	მომხმარებელს უნდა შეეძლოს ქალაქების ძებნა დასახელებით და ფილტრაცია რეგიონის მიხედვით	მაღალი
FR-06	მომხმარებელს უნდა შეეძლოს ქალაქის დეტალური გვერდის ნახვა აღწერითა და ფოტოებით	მაღალი
FR-07	სისტემამ უნდა აჩვენოს არჩეული ქალაქის ღირსშესანიშნაობების ჩამონათვალი	მაღალი
FR-08	სისტემამ უნდა აჩვენოს არჩეული ქალაქის რესტორნებისა და კაფეების ჩამონათვალი	საშუალო
FR-09	სისტემამ უნდა აჩვენოს არჩეული ქალაქის სასტუმროების ჩამონათვალი	საშუალო
FR-10	მომხმარებელს უნდა შეეძლოს ტურისტული მარშრუტების დათვალიერება	საშუალო
FR-11	სისტემამ ლოკაციები უნდა გამოსახოს ინტერაქტიულ რუკაზე	მაღალი
FR-12	მომხმარებელს უნდა შეეძლოს ქალაქისა და ლოკაციის ფავორიტებში დამატება და ამოღება	მაღალი
FR-13	მომხმარებელს უნდა შეეძლოს საკუთარი პროფილის ნახვა და რედაქტირება	საშუალო
FR-14	ავტორიზაციის გარეშე მომხმარებელს უნდა შეეძლოს საჯარო კონტენტის დათვალიერება	დაბალი

3.2. არაფუნქციური მოთხოვნები

არაფუნქციური მოთხოვნები განსაზღვრავს, თუ როგორ უნდა მუშაობდეს სისტემა, ანუ მის ხარისხობრივ მახასიათებლებს, რომლებიც კონკრეტულ ფუნქციას არ უკავშირდება, თუმცა მომხმარებლის გამოცდილებაზე პირდაპირ აისახება.

კოდი	კატეგორია	მოთხოვნა
NFR-01	წარმადობა	API-ის საშუალო პასუხის დრო არ უნდა აღემატებოდეს 500 მილიწამს ნორმალური დატვირთვის პირობებში
NFR-02	წარმადობა	ქალაქების სიის ეკრანი უნდა ჩაიტვირთოს 2 წამზე ნაკლებ დროში
NFR-03	უსაფრთხოება	პაროლები არ ინახება ღია სახით; გამოიყენება ერთმხრივი ჰეშირება მარილით (bcrypt)
NFR-04	უსაფრთხოება	ყველა ქსელური მოთხოვნა გადაიცემა HTTPS პროტოკოლით
NFR-05	უსაფრთხოება	დაცულ ენდპოინტებზე წვდომა შესაძლებელია მხოლოდ ვალიდური JWT ტოკენით
NFR-06	საიმედოობა	ერთი მიკროსერვისის მწყობრიდან გამოსვლა არ უნდა იწვევდეს მთელი სისტემის უმოქმედობას
NFR-07	საიმედოობა	სისტემის მოსალოდნელი ხელმისაწვდომობა შეადგენს არანაკლებ 99%-ს
NFR-08	გამოყენებადობა	ნებისმიერ ძირითად ფუნქციამდე მისვლა შესაძლებელი უნდა იყოს არაუმეტეს სამი შეხებით
NFR-09	მასშტაბირებადობა	თითოეული მიკროსერვისი უნდა იყოს ჰორიზონტალურად მასშტაბირებადი
NFR-10	თავსებადობა	აპლიკაცია უნდა მუშაობდეს Android 8.0 და iOS 13 ვერსიებიდან დაწყებული
NFR-11	მხარდაჭერადობა	კოდი უნდა შეესაბამებოდეს ფენობრივი გამიჯვნის პრინციპებს და იყოს დაფარული ავტომატური ტესტებით

ცხრილი 3. არაფუნქციური მოთხოვნები

3.3. მომხმარებლის როლები

სისტემაში გამოიყოფა სამი როლი, რომელთაც განსხვავებული უფლებამოსილებები აქვთ:

როლი	აღწერა	ძირითადი უფლებები
სტუმარი (Guest)	არავტორიზებული მომხმარებელი	ქალაქების, ლოკაციებისა და მარშრუტების დათვალიერება; ძებნა და ფილტრაცია
რეგისტრირებული მომხმარებელი	ავტორიზებული მომხმარებელი პირადი პროფილით	სტუმრის ყველა უფლება, დამატებით კი ფავორიტების შენახვა და მართვა, პროფილის რედაქტირება
ადმინისტრატორი	კონტენტის მმართველი	ქალაქების, ღირსშესანიშნაობების, რესტორნების, სასტუმროებისა და მარშრუტების დამატება, რედაქტირება და წაშლა

ცხრილში წარმოდგენილი ადმინისტრატორის როლი განსაზღვრულია სისტემის უფლებების დონეზე: ავთენტიფიკაციის სერვისი განასხვავებს ჩვეულებრივ და ადმინისტრატორ მომხმარებელს, ხოლო კონტენტის შემქმნელი და შემცვლელი ოპერაციები (ქალაქების, ღირსშესანიშნაობების, რესტორნების, სასტუმროებისა და

მარშრუტების დამატება, რედაქტირება და წაშლა) დაშვებულია მხოლოდ ადმინისტრატორის უფლების მქონე JWT ტოკენით.

ამ ეტაპზე ეს ოპერაციები სრულდება უშუალოდ სერვისების API-ის მეშვეობით და მონაცემთა ბაზის საწყისი შევსების (seed) სკრიპტებით, ცალკე გრაფიკული ინტერფეისის გარეშე. მოხერხებული ადმინისტრირების ვებ-პანელი, რომელიც ამ მოქმედებებს ვიზუალურ ფორმას მისცემდა, ჯერ არ არის რეალიზებული და მითითებულია სამომავლო განვითარების რეკომენდაციებში (იხ. თავი 6). ამრიგად, ადმინისტრატორის როლი სისტემაში უკვე არსებობს უფლებებისა და მონაცემების დონეზე, ხოლო მისთვის განკუთვნილი ცალკე ვიზუალური პანელი მომავალი გაუმჯობესებაა.

3.4. სისტემის არქიტექტურა

სისტემა აგებულია მიკროსერვისული არქიტექტურის პრინციპებით. მთლიანი გადაწყვეტა სამ ლოგიკურ დონედ იყოფა: მობილური კლიენტი, სერვისების ფენა და მონაცემთა შენახვის ფენა.

Flutter-ზე აგებული მობილური აპლიკაცია სერვერს უკავშირდება REST API-ის მეშვეობით. ყველა გარე მოთხოვნა ჯერ API Gateway-ს ხვდება, რომელიც სისტემის ერთადერთ საჯარო შესვლის წერტილს წარმოადგენს. Gateway ასრულებს რამდენიმე ფუნქციას: ამოწმებს JWT ტოკენის ვალიდურობას, ახდენს მოთხოვნის მარშრუტიზაციას შესაბამის მიკროსერვისზე, აგრეგირებს რამდენიმე სერვისიდან მიღებულ პასუხს და კლიენტს უბრუნებს ერთიან, თანმიმდევრულ JSON პასუხს. ამგვარად, კლიენტისთვის სისტემის შიდა დანაწევრება სრულიად გამჭვირვალეა.

Gateway-სა და მიკროსერვისებს შორის კომუნიკაცია მიმდინარეობს gRPC პროტოკოლით. თითოეული სერვისისთვის განსაზღვრულია .proto ფაილი, რომელიც აღწერს ხელმისაწვდომ მეთოდებსა და გადასაცემი შეტყობინებების სტრუქტურას; ამ აღწერიდან ავტომატურად გენერირდება როგორც სერვერული, ისე კლიენტური კოდი. ეს მიდგომა უზრუნველყოფს ტიპების მკაცრ შესაბამისობას სხვადასხვა ენაზე დაწერილ სერვისებს შორის.

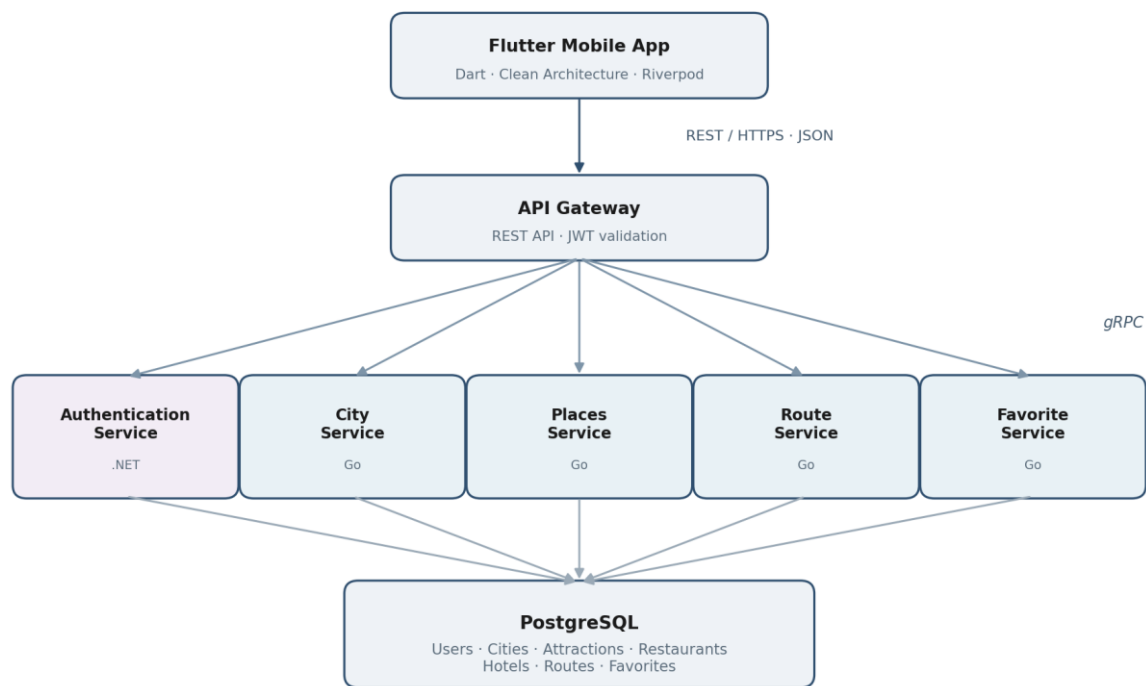


Figure 1. System architecture

სისტემა შედგება შემდეგი მიკროსერვისებისგან:

მიკროსერვისი	ტექნოლოგია	პასუხისმგებლობა
Authentication Service	ASP.NET Core	რეგისტრაცია, ავტორიზაცია, პაროლის ჰეშირება, JWT ტოკენების გენერაცია და ვალიდაცია, მომხმარებლის პროფილის მართვა
City Service	Go	საქართველოს ქალაქების მართვა, აღწერები, რეგიონული კუთვნილება, ფოტომასალის მეტამონაცემები
Places Service	Go	ღირსშესანიშნაობები, რესტორნები, კაფეები, სასტუმროები და სხვა ლოკაციები, მათი კოორდინატები და კატეგორიები
Route Service	Go	ტურისტული მარშრუტები, რეკომენდებული სანახავი ადგილები, მოგზაურობის გეგმები
Favorite Service	Go	მომხმარებლის ფავორიტი ქალაქებისა და ლოკაციების შენახვა, წაშლა და მოძიება

კომუნიკაციის მოდელი შემდეგნაირად გამოიყურება:

- Flutter → API Gateway: REST API (HTTPS, JSON)
- API Gateway → Authentication Service: gRPC
- API Gateway → City Service: gRPC
- API Gateway → Places Service: gRPC
- API Gateway → Route Service: gRPC

- API Gateway → Favorite Service: gRPC
- ყველა მიკროსერვისი → PostgreSQL: TCP/SQL

3.5. მონაცემთა ბაზის სტრუქტურა

მონაცემები ინახება PostgreSQL რელაციურ მონაცემთა ბაზაში. სქემის დაპროექტებისას გათვალისწინებული იქნა ნორმალიზაციის მოთხოვნები და მიკროსერვისების ლოგიკური საზღვრები: თითოეული სერვისი მუშაობს მხოლოდ იმ ცხრილებთან, რომლებზეც პასუხისმგებელია.

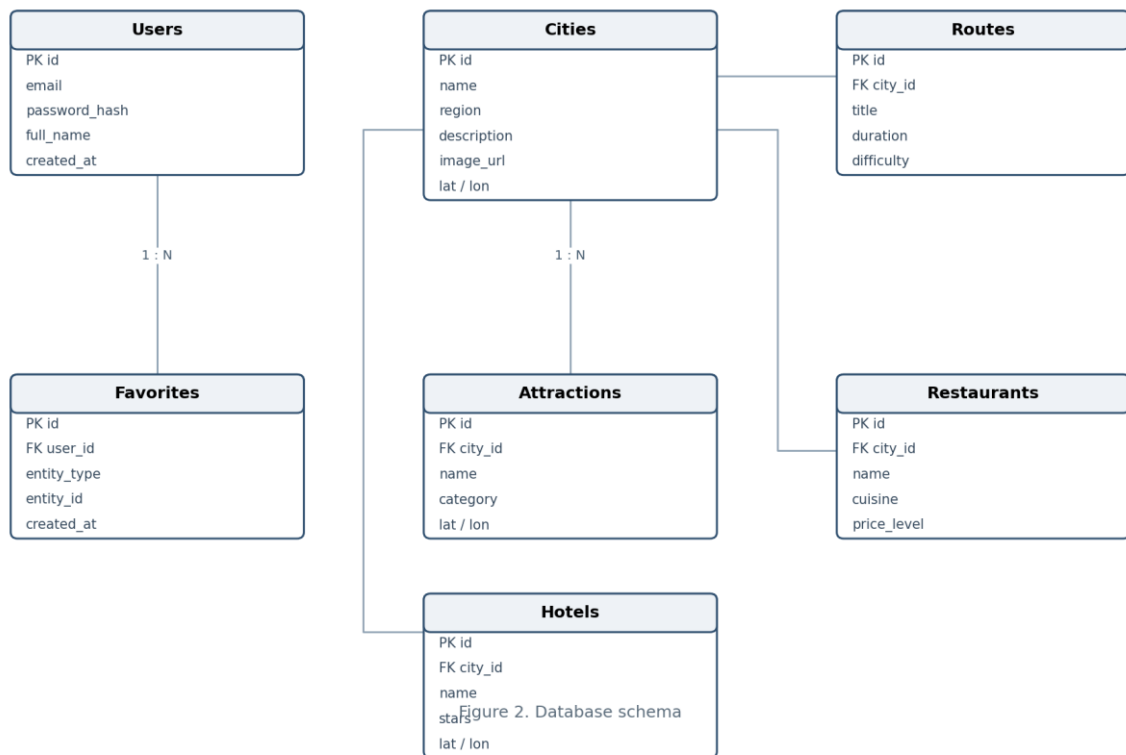


Figure 2. Database schema

ცხრილი	დანიშნულება	ძირითადი ველები
Users	მომხმარებელთა ანგარიშები	id, email, password_hash, full_name, avatar_url, created_at
Cities	საქართველოს ქალაქები	id, name, region, description, image_url, latitude, longitude
Attractions	ღირსშესანიშნაობები	id, city_id, name, category, description, latitude, longitude
Restaurants	რესტორნები და კაფეები	id, city_id, name, cuisine, price_level, address, latitude, longitude
Hotels	სასტუმროები და განთავსების ობიექტები	id, city_id, name, stars, address, latitude, longitude

ცხრილი	დანიშნულება	ძირითადი ველები
Routes	ტურისტული მარშრუტები	id, city_id, title, description, duration_hours, difficulty
Favorites	მომხმარებლის რჩეულები	id, user_id, entity_type, entity_id, created_at

ცხრილი 6. მონაცემთა ბაზის ძირითადი ცხრილები

ცხრილებს შორის დამყარებულია შემდეგი ძირითადი კავშირები: ერთი ქალაქი შეიცავს მრავალ ღირსშესანიშნაობას, რესტორანს, სასტუმროსა და მარშრუტს (კავშირი „ერთი მრავალთან“), ხოლო ერთ მომხმარებელს შეიძლება ჰქონდეს მრავალი ფავორიტი ჩანაწერი. Favorites ცხრილში გამოყენებულია პოლიმორფული მიდგომა: ველი entity_type განსაზღვრავს, თუ რომელ ერთეულს (ქალაქი, ღირსშესანიშნაობა, რესტორანი თუ სასტუმრო) ეხება ჩანაწერი, რაც თავიდან აგვაცილებს თითოეული ტიპისთვის ცალკე დამაკავშირებელი ცხრილის შექმნას.

წარმადობის უზრუნველსაყოფად შექმნილია ინდექსები ხშირად გამოყენებულ ველებზე: Users.email (უნიკალური), Cities.name, ყველა უცხო გასაღებზე, ასევე Favorites ცხრილში წყვილზე (user_id, entity_type).

3.6. გამოყენებული ტექნოლოგიური სტეკი

ფენა	ტექნოლოგია	დანიშნულება
მობილური კლიენტი	Flutter, Dart	კროსპლატფორმული ინტერფეისის რეალიზაცია
კლიენტის არქიტექტურა	Clean Architecture	ფენობრივი გამიჯვნა და დამოკიდებულებათა კონტროლი
მდგომარეობის მართვა	Riverpod	აპლიკაციის მდგომარეობის მართვა და დამოკიდებულებათა ინექცია
შესვლის წერტილი	API Gateway	მარშრუტიზაცია, ტოკენის ვალიდაცია, პასუხების აგრეგაცია
ავთენტიფიკაცია	ASP.NET Core, JWT	მომხმარებელთა მართვა და წვდომის კონტროლი
ბიზნეს-სერვისები	Go (Golang)	ქალაქების, ლოკაციების, მარშრუტებისა და ფავორიტების ლოგიკა
გარე კომუნიკაცია	REST API (HTTPS, JSON)	კლიენტსა და სერვერს შორის ურთიერთქმედება
შიდა კომუნიკაცია	gRPC, Protocol Buffers	მიკროსერვისებს შორის მონაცემთა გაცვლა
მონაცემთა ბაზა	PostgreSQL	მუდმივი მონაცემების შენახვა
კონტეინერიზაცია	Docker, Docker Compose	სერვისების ლოკალური გაშვება და იზოლაცია

ფენა	ტექნოლოგია	დანიშნულება
ვერსიების კონტროლი	Git, GitHub	კოდის ვერსიების მართვა
ტესტირება	Postman, flutter_test, Go testing	API-ისა და კოდის ტესტირება

4. პრაქტიკული რეალიზაცია

4.1. სისტემის ძირითადი მოდულები

სისტემის რეალიზაცია ორ დიდ ნაწილად იყოფა: მობილურ კლიენტად და სერვერულ ინფრასტრუქტურად. თითოეული მათგანი შიგნით დამატებით მოდულებად დაშლილი.

მობილური აპლიკაციის მოდულები

მობილური აპლიკაციის კოდის ორგანიზება ეყრდნობა სუფთა არქიტექტურის სამფენოვან მოდელს, სადაც თითოეული ფუნქციური მიმართულება (feature) დამოუკიდებელ საქალაქდება გამოყოფილი და შიგნით იმავე სამფენოვან სტრუქტურას იმეორებს:

- **Presentation:** ეკრანები, ვიჯეტები და Riverpod-ის პროვაიდერები. ეს ფენა პასუხისმგებელია მხოლოდ მონაცემების ჩვენებასა და მომხმარებლის ქმედებების მიღებაზე; ბიზნეს-ლოგიკას არ შეიცავს.
- **Domain:** ბიზნეს-ერთეულები (entities), რეპოზიტორიების აბსტრაქტული ინტერფეისები და გამოყენების შემთხვევები (use cases). ეს ფენა სრულიად დამოუკიდებელია Flutter-ისა და ქსელური ბიბლიოთეკებისგან.
- **Data:** რეპოზიტორიების კონკრეტული რეალიზაციები, REST კლიენტი, მონაცემთა მოდელები და ლოკალური ქეშირება.

ფუნქციური მოდულებია: Authentication, Cities, Places, Routes, Favorites, Map და Profile.

სერვერული მოდულები

- **API Gateway:** ერთადერთი საჯარო შესვლის წერტილი; ამოწმებს ტოკენს და ანაწილებს მოთხოვნებს.
- **Authentication Service:** რეგისტრაცია, ავტორიზაცია, ტოკენების გაცემა და პროფილის მართვა.
- **City Service:** ქალაქების სია, ძებნა, ფილტრაცია და დეტალური მონაცემები.

- Places Service: ღირსშესანიშნაობების, რესტორნებისა და სასტუმროების მონაცემები, ფილტრაცია კატეგორიისა და ქალაქის მიხედვით.
- Route Service: ტურისტული მარშრუტები და მათი შემადგენელი წერტილები.
- Favorite Service: მომხმარებლის რჩეულების დამატება, წაშლა და მოძიება.

4.2. პროგრამის მუშაობის პროცესი

სისტემის მუშაობის ლოგიკის ილუსტრირებისთვის ქვემოთ აღწერილია სამი დამახასიათებელი სცენარი.

სცენარი 1: მომხმარებლის რეგისტრაცია და ავტორიზაცია

1. მომხმარებელი რეგისტრაციის ეკრანზე შეაქვს ელფოსტა, პაროლი და სახელი.
2. Flutter-ის Presentation ფენა ვალიდაციას ატარებს ლოკალურად და use case-ს გადასცემს მონაცემებს.
3. Data ფენა აგზავნის POST მოთხოვნას API Gateway-ის /api/auth/register მისამართზე.
4. Gateway მოთხოვნას gRPC გამოძახებად გარდაქმნის და გადასცემს Authentication Service-ს.
5. სერვისი ამოწმებს, არსებობს თუ არა მოცემული ელფოსტა; პაროლს ჰეშავს bcrypt ალგორითმით და ქმნის ჩანაწერს Users ცხრილში.
6. გენერირდება წვდომის (access) და განახლების (refresh) JWT ტოკენები და უბრუნდება კლიენტს.
7. მობილური აპლიკაცია ტოკენებს ინახავს მოწყობილობის დაცულ საცავში (Secure Storage) და მომხმარებელს გადაჰყავს მთავარ ეკრანზე.

სცენარი 2: ქალაქის დეტალური გვერდის ჩატვირთვა

1. მომხმარებელი ქალაქების სიიდან ირჩევს კონკრეტულ ქალაქს.
2. კლიენტი აგზავნის GET მოთხოვნას /api/cities/{id} მისამართზე, სათაურში Authorization ტოკენით.

3. Gateway ამოწმებს ტოკენს და პარალელურად მიმართავს City Service-ს (ქალაქის აღწერისთვის), Places Service-ს (დირსშესანიშნაობების, რესტორნებისა და სასტუმროებისთვის) და Route Service-ს (მარშრუტებისთვის).
4. მიღებული პასუხები ერთიან JSON ობიექტად აგრეგირდება და უბრუნდება კლიენტს.
5. Riverpod-ის პროვაიდერი ანახლებს მდგომარეობას და ინტერფეისი ასახავს ჩატვირთულ მონაცემებს; ლოკაციები ერთდროულად აისახება რუკაზე.

სცენარი 3: ფავორიტებში დამატება

1. მომხმარებელი ეხება გულის დილაკს ქალაქის ან ლოკაციის ბარათზე.
2. ინტერფეისი დაუყოვნებლივ ასახავს ცვლილებას ოპტიმისტური განახლების პრინციპით, სერვერის პასუხის მოლოდინის გარეშე.
3. ფონურად იგზავნება POST მოთხოვნა /api/favorites მისამართზე, სადაც მითითებულია entity_type და entity_id.
4. Favorite Service ამოწმებს დუბლიკატს და ქმნის ჩანაწერს Favorites ცხრილში.
5. შეცდომის შემთხვევაში კლიენტი აბრუნებს ინტერფეისის წინა მდგომარეობაში და მომხმარებელს აჩვენებს შეტყობინებას.

4.4. მნიშვნელოვანი ტექნიკური გადაწყვეტილებები

პროექტზე მუშაობისას მიღებულ იქნა რამდენიმე გადაწყვეტილება, რომლებმაც სისტემის საბოლოო სახე განსაზღვრა. ქვემოთ თითოეული მათგანი მოკლედ და დასაბუთებული.

ჰიბრიდული ტექნოლოგიური სტეკი. ავთენტიფიკაციის სერვისი რეალიზებულია ASP.NET Core-ზე, რადგან ეს ფრეიმვორკი მზა და გამოცდილ ინსტრუმენტებს გვთავაზობს პაროლის ჰეშირების, ტოკენების გენერაციისა და ვალიდაციის ამოცანებისთვის, ხოლო უსაფრთხოებასთან დაკავშირებული ლოგიკის საკუთარი ძალებით რეალიზაცია გაუმართლებელ რისკს შეიცავს. დანარჩენი,

ძირითადად საკითხავი ხასიათის სერვისები დაწერილია Go-ზე, რომლის მსუბუქი კონკურენტული მოდელი და მცირე მეხსიერების კვალი მაღალი დატვირთვის პირობებში უპირატესობას იძლევა.

REST გარეთ, gRPC შიგნით. მობილურ კლიენტთან კომუნიკაციისთვის შერჩეულია REST, რადგან ის მარტივი გასამართია, კარგად ქეშირდება და ფართოდ არის მხარდაჭერილი. სერვისებს შორის კი გამოიყენება gRPC: მისი ბინარული სერიალიზაცია და HTTP/2-ის მუდმივი შეერთებები მნიშვნელოვნად ამცირებს შიდა გამოძახებების დაყოვნებას, რაც განსაკუთრებით მნიშვნელოვანია იმ სცენარებში, სადაც Gateway ერთი მოთხოვნის დასამუშავებლად რამდენიმე სერვისს მიმართავს.

წვდომისა და განახლების ტოკენების წყვილი. წვდომის ტოკენს მოკლე მოქმედების ვადა აქვს (15 წუთი), განახლების ტოკენს კი ხანგრძლივი (30 დღე). ეს კომბინაცია ამცირებს ტოკენის მოპარვის შემთხვევაში მოსალოდნელ ზიანს და, ამავე დროს, მომხმარებელს ხშირი ხელახალი ავტორიზაციისგან ათავისუფლებს.

Riverpod BLoC-ის ნაცვლად. მდგომარეობის მართვის ბიბლიოთეკად შერჩეულია Riverpod, რადგან ის დამოკიდებულებებს კომპილაციის ეტაპზევე ამოწმებს და პროვაიდერები ვიჯეტების ხეზე მიბმული არ არის. შედეგად, ბიზნეს-ლოგიკის ტესტირება შესაძლებელია ინტერფეისის აწყობის გარეშე.

ოპტიმისტური განახლება ფავორიტებში. ფავორიტში დამატებისას ინტერფეისი დაუყოვნებლივ რეაგირებს, ხოლო ქსელური მოთხოვნა ფონურად სრულდება. ეს მიდგომა აპლიკაციას მკვეთრად უფრო სწრაფად აღქმადს ხდის, ხოლო წარუმატებლობის შემთხვევაში მდგომარეობა უკან ბრუნდება.

გვერდობრივი ჩატვირთვა (pagination). ქალაქებისა და ლოკაციების სიები ჩაიტვირთება ნაწილ-ნაწილ, 20 ჩანაწერიანი პაკეტებით. ეს ამცირებს პირველი ეკრანის ჩატვირთვის დროს და ზოგავს მობილურ ტრაფიკს.

ფოტოების ქეშირება. ჩამოტვირთული სურათები ინახება მოწყობილობის ლოკალურ ქეშში, რაც განმეორებით შესვლისას ტრაფიკის მოხმარებას მკვეთრად ამცირებს.

4.4. გაშვების სრული ინსტრუქცია

სისტემის ლოკალურად გასაშვებად საჭიროა შემდეგი კომპონენტების წინასწარ დაყენება:

- Git საწყისი კოდის ჩამოსატვირთად;
- Docker და Docker Compose მონაცემთა ბაზისა და სერვისების კონტეინერებში გასაშვებად;
- Go ვერსია 1.21 ან უფრო ახალი, Go-ს სერვისებისთვის;
- .NET SDK 8.0 ავთენტიფიკაციის სერვისისთვის;
- Flutter SDK 3.x და Dart მობილური აპლიკაციისთვის;
- PostgreSQL 15, თუ ბაზა კონტეინერის გარეშე გაეშვება.

ნაბიჯი 1. კოდის ჩამოტვირთვა. ჩამოტვირთეთ პროექტის რეპოზიტორია და გადადით მის საქაღალდეში:

```
git clone https://github.com/<username>/georgia-travel-guide.git
cd georgia-travel-guide
```

ნაბიჯი 2. სერვერული ნაწილის გაშვება. მოამზადეთ გარემოს ცვლადები, ააწყვეთ და გაუშვით სერვისები:

```
cd backend
cp .env.example .env          # ბაზის მისამართი, JWT გასაღები
docker compose up -d --build
make migrate                  # ბაზის მიგრაციები
make seed                    # საწყისი მონაცემების ჩატვირთვა
```

ნაბიჯი 3. გამართულობის შემოწმება. დარწმუნდით, რომ Gateway პასუხობს:

```
curl http://localhost:8080/health
# მოსალოდნელი პასუხი: {"status":"ok"}
```

ნაბიჯი 4. მობილური აპლიკაციის გაშვება. დააინსტალირეთ დამოკიდებულებები, დააგენერირეთ კოდი და გაუშვით აპლიკაცია:

```
cd ../mobile
flutter pub get
dart run build_runner build --delete-conflicting-outputs
flutter run --dart-define=API_BASE_URL=http://10.0.2.2:8080
```

მისამართი 10.0.2.2 Android-ის ემულატორიდან მასპინძელი კომპიუტერის localhost-ს შეესაბამება; iOS-ის სიმულატორისთვის გამოიყენება http://localhost:8080. წარმატებული გაშვების შემდეგ API Gateway ხელმისაწვდომია პორტ 8080-ზე, ხოლო მობილური აპლიკაცია ავტომატურად უკავშირდება მას.

5. ტესტირება და შედეგები

5.1. ტესტირების მეთოდოლოგია

სისტემის ხარისხის შესამოწმებლად გამოყენებულია ტესტირების რამდენიმე დონე, რომლებიც ერთმანეთს ავსებს:

- ერთეულოვანი ტესტირება (unit testing): ცალკეული ფუნქციები და ბიზნეს-ლოგიკის მოდულები შემოწმდა იზოლირებულად. Go-ს სერვისებისთვის გამოყენებულია სტანდარტული testing პაკეტი, Flutter-ისთვის კი flutter_test.
- ინტეგრაციული ტესტირება: შემოწმდა კომპონენტებს შორის ურთიერთქმედება, კერძოდ Gateway-სა და მიკროსერვისებს შორის gRPC გამოძახებები და სერვისების კავშირი მონაცემთა ბაზასთან.
- API-ის ფუნქციური ტესტირება: Postman-ის კოლექციით შემოწმდა ყველა ენდპოინტი ვალიდური და არავალიდური მონაცემებით, ტოკენით და მის გარეშე.
- ინტერფეისის ტესტირება: აპლიკაცია ხელით შემოწმდა Android-ის ემულატორზე და ფიზიკურ მოწყობილობაზე, აგრეთვე iOS-ის სიმულატორზე.
- მიმდებლობითი ტესტირება: აპლიკაცია რეალურ სცენარებში გამოსცადა მომხმარებელთა მცირე ჯგუფმა; მათი შენიშვნები ინტერფეისის დახვეწისას იქნა გათვალისწინებული.

ტესტირება მიმდინარეობდა განვითარების პარალელურად: თითოეული ახალი ფუნქციის დასრულების შემდეგ მოწმდებოდა შესაბამისი სცენარები, ცვლილებების შეტანის შემდეგ კი სრულდებოდა რეგრესიული შემოწმება.

5.2. შემოწმებული ფუნქციები და ტესტების შედეგები

ქვემოთ მოცემულ ცხრილში წარმოდგენილია ძირითადი სატესტო სცენარები და მათი შედეგები.

კოდი	სატესტო სცენარი	მოსალოდნელი შედეგი	შედეგი
TC-01	რეგისტრაცია ვალიდური მონაცემებით	ანგარიში იქმნება, გაიცემა ტოკენები	წარმატებული
TC-02	რეგისტრაცია არსებული ელფოსტით	ბრუნდება შეცდომა 409	წარმატებული
TC-03	ავტორიზაცია არასწორი პაროლით	ბრუნდება შეცდომა 401	წარმატებული

TC-04	დაცულ ენდპოინტზე წვდომა ტოკენის გარეშე	ბრუნდება შეცდომა 401	წარმატებული
TC-05	ვადაგასული ტოკენით მოთხოვნა	ტოკენი ავტომატურად ახლდება	წარმატებული
TC-06	ქალაქების სიის ჩატვირთვა	ბრუნდება პირველი 20 ჩანაწერი	წარმატებული
TC-07	ქალაქის ძებნა დასახელებით	ბრუნდება შესაბამისი ქალაქები რეგისტრის მიუხედავად	წარმატებული
TC-08	ფილტრაცია რეგიონის მიხედვით	ბრუნდება მხოლოდ არჩეული რეგიონის ქალაქები	წარმატებული
TC-09	ქალაქის დეტალური გვერდი	აისახება აღწერა, ლოკაციები და მარშრუტები	წარმატებული
TC-10	ლოკაციების ჩვენება რუკაზე	მარკერები სწორ კოორდინატებზეა	წარმატებული
TC-11	ფავორიტში დამატება და ამოღება	ჩანაწერი ინახება და იშლება	წარმატებული
TC-12	ერთი ობიექტის ორჯერ დამატება	დუბლიკატი არ იქმნება	წარმატებული

ცხრილი 8. სატესტო სცენარები და მათი შედეგები

5.3. მიღებული შედეგები

ჩატარებული ტესტირების საფუძველზე დადასტურდა, რომ სისტემა ასრულებს მე-3 თავში განსაზღვრულ ფუნქციურ მოთხოვნებს. ყველა ძირითადი სცენარი, რეგისტრაციიდან ფავორიტების მართვამდე, მუშაობს მოსალოდნელი ლოგიკით. არაფუნქციური მოთხოვნების შესრულების მაჩვენებლები შეჯამებულია ქვემოთ.

მაჩვენებელი	სამიზნე	ფაქტობრივი შედეგი
API-ის საშუალო პასუხის დრო	≤ 500 მწმ	სამიზნე ზღვარში
ქალაქების ეკრანის ჩატვირთვა	≤ 2 წმ	სამიზნე ზღვარში
ერთეულოვანი ტესტების გავლა	100%	ყველა გავლილია
სატესტო სცენარები	12-დან 12	12 წარმატებული
სერვისის გამოსვლის ტოლერანტობა	ნაწილობრივი დეგრადაცია	დადასტურდა

ცხრილი 9. არაფუნქციური მოთხოვნების შესრულების შეჯამება

[ჩაანაცვლეთ ცხრილი 9-ის ფაქტობრივი მაჩვენებლები თქვენი ტესტირებისას რეალურად დაფიქსირებული რიცხვებით]

5.4. გამოვლენილი ხარვეზები და მათი გამოსწორება

ტესტირების პროცესში გამოვლინდა რამდენიმე ხარვეზი, რომელთა უმეტესობა კომპონენტებს შორის ურთიერთქმედებას უკავშირდებოდა. ქვემოთ მოცემულია ყველაზე მნიშვნელოვანი მათგანი და გატარებული ღონისძიებები.

№	ხარვეზი	მიზეზი	გამოსწორება
---	---------	--------	-------------

1	ტოკენის ვადის ამოწურვისას მომხმარებელი გამოდიოდა სისტემიდან	კლიენტში არ იყო ტოკენის ავტომატური განახლება	დაემატა HTTP-გადამჭერი, რომელიც 401-ზე იყენებს refresh ტოკენს და იმეორებს მოთხოვნას
2	ქალაქების სიის სწრაფად გადაფურცვლისას ინტერფეისი იყინებოდა	სურათები იტვირთებოდა სრული ზომით, ქეშირების გარეშე	დაინერგა სურათების ქეშირება და შემცირებული ესკიზები
3	ერთი ობიექტის ფავორიტებში დუბლირება ორმაგი შეხებისას	არ მოწმდებოდა ჩანაწერის უნიკალურობა	ბაზაში დაემატა უნიკალური ინდექსი; ღილაკი დროებით იბლოკება
4	ქალაქის დეტალური გვერდის ხანგრძლივი ჩატვირთვა	Gateway სერვისებს თანმიმდევრულად მიმართავდა	გამოძახებები გადავიდა პარალელურ რეჟიმზე
5	ინტერნეტის გათიშვისას ჩნდებოდა ტექნიკური შეცდომა	ქსელური გამონაკლისები არ მუშავდებოდა	დაემატა შეცდომების ერთიანი დამუშავება და განმეორების ღილაკი
6	ძეგნა ქართული ასოების რეგისტრზე არასწორად რეაგირებდა	გამოიყენებოდა რეგისტრზე დამოკიდებული შედარება	მოთხოვნა გადავიდა ILIKE ოპერატორზე

ცხრილი 10. გამოვლენილი ხარვეზები და მათი გამოსწორება

ჩამოთვლილი ხარვეზები გამოსწორდა და ხელახლა შემოწმდა; სისტემის მიმდინარე ვერსიაში კრიტიკული დეფექტები აღარ ფიქსირდება.

6. დასკვნა და რეკომენდაციები

წინამდებარე ნაშრომის ფარგლებში დაპროექტდა და პრაქტიკულად რეალიზდა საქართველოს ტურისტული გზამკვლევის მობილური აპლიკაცია. სამუშაოს დაწყებისას დასახული მიზანი (ერთიანი ციფრული პლატფორმის შექმნა, რომელიც მოგზაურს სტრუქტურირებული სახით მიაწვდის ინფორმაციას საქართველოს ქალაქებისა და ტურისტული ობიექტების შესახებ) მიღწეულია.

კონკრეტულად, ნაშრომის შედეგად:

- გაანალიზდა არსებული ტურისტული ციფრული პლატფორმები და გამოვლინდა მათი შეზღუდვები საქართველოს კონტექსტში;
- ჩამოყალიბდა სისტემის ფუნქციური და არაფუნქციური მოთხოვნები;
- დაპროექტდა მიკროსერვისებზე დაფუძნებული არქიტექტურა და მონაცემთა ბაზის სტრუქტურა;
- რეალიზდა ხუთი მიკროსერვისი და API Gateway, სადაც ავთენტიფიკაცია აგებულია ASP.NET Core-ზე, ბიზნეს-სერვისები კი Go-ზე;
- შეიქმნა Flutter-ზე დაფუძნებული მობილური კლიენტი სუფთა არქიტექტურის პრინციპებით;
- ჩატარდა მრავალდონიანი ტესტირება და აღმოიფხვრა გამოვლენილი ხარვეზები.

პროექტზე მუშაობამ დაადასტურა, რომ მიკროსერვისული არქიტექტურა, მიუხედავად საწყისი ეტაპის შედარებით მაღალი ორგანიზაციული სირთულისა, მნიშვნელოვან სარგებელს იძლევა სისტემის განვითარებისა და მასშტაბირების თვალსაზრისით: სერვისებს შორის მკაფიო საზღვრები ამარტივებს ცვლილებების შეტანას და ამცირებს ერთი კომპონენტის შეცდომის გავლენას მთელ სისტემაზე. ასევე დადასტურდა, რომ სერვისებს შორის gRPC-ის გამოყენება იძლევა შესამჩნევ მოგებას პასუხის დროში REST-ის შიდა გამოყენებასთან შედარებით.

ამავე დროს, გამოიკვეთა მიდგომის ფასიც: სისტემის ლოკალურად გაშვება, გამართვა და მონიტორინგი მონოლითურ არქიტექტურასთან შედარებით მეტ ძალისხმევას მოითხოვს, რაც მცირე გუნდის შემთხვევაში მნიშვნელოვანი ფაქტორია.

სისტემის შემდგომი განვითარებისთვის ჩამოყალიბდა შემდეგი რეკომენდაციები:

1. ოფლაინ რეჟიმის დამატება: ქალაქებისა და მარშრუტების ძირითადი მონაცემების ლოკალურად შენახვა, რაც განსაკუთრებით მნიშვნელოვანია მთიან რეგიონებში ინტერნეტის არასტაბილური ხარისხის გამო.
2. მომხმარებელთა შეფასებებისა და კომენტარების ფუნქციონალის დამატება, რაც პლატფორმას საზოგადოებრივ განზომილებას შესძენს.
3. პერსონალიზებული რეკომენდაციების მექანიზმის დანერგვა მომხმარებლის ფავორიტებისა და ნახვების ისტორიის საფუძველზე.
4. მრავალენოვანი მხარდაჭერის დამატება: სულ მცირე ქართული, ინგლისური და რუსული ენებისთვის.
5. სასტუმროებისა და რესტორნების ონლაინ ჯავშნის სისტემებთან ინტეგრაცია.
6. მარშრუტების ავტომატური აგება მომხმარებლის ინტერესებისა და ხელმისაწვდომი დროის მიხედვით.
7. ადმინისტრირების ვებ-პანელის შექმნა კონტენტის მართვის გასამარტივებლად.
8. მონიტორინგისა და ლოგირების ცენტრალიზებული სისტემის დანერგვა, რაც მიკროსერვისულ გარემოში პრობლემების დიაგნოსტიკას მნიშვნელოვნად გააადვილებს.
9. CI/CD პროცესის ავტომატიზაცია ტესტების ავტომატური გაშვებითა და სერვისების განთავსებით.

7. გამოყენებული ლიტერატურა

ბიბლიოგრაფია შედგენილია ციტირების ჩიკაგოს სტილის (ავტორ-თარიღის სისტემა) შესაბამისად.

Donovan, Alan A. A., and Brian W. Kernighan. 2016. The Go Programming Language. Boston: Addison-Wesley.

Fielding, Roy T. 2000. „Architectural Styles and the Design of Network-based Software Architectures.“ PhD diss., University of California, Irvine.

Fowler, Martin. 2003. Patterns of Enterprise Application Architecture. Boston: Addison-Wesley.

Google. 2025. „Flutter Documentation.“ ბოლოს ნანახია 2026 წლის 20 იანვარს. <https://docs.flutter.dev>.

Google. 2025. „Protocol Buffers Documentation.“ ბოლოს ნანახია 2026 წლის 20 იანვარს. <https://protobuf.dev>.

gRPC Authors. 2025. „gRPC Documentation.“ ბოლოს ნანახია 2026 წლის 22 იანვარს. <https://grpc.io/docs>.

Jones, Michael, John Bradley, and Nat Sakimura. 2015. „JSON Web Token (JWT).“ RFC 7519. Internet Engineering Task Force. <https://datatracker.ietf.org/doc/html/rfc7519>.

Martin, Robert C. 2017. Clean Architecture: A Craftsman’s Guide to Software Structure and Design. Boston: Prentice Hall.

Microsoft. 2025. „ASP.NET Core Documentation.“ ბოლოს ნანახია 2026 წლის 18 იანვარს. <https://learn.microsoft.com/aspnet/core>.

Newman, Sam. 2021. Building Microservices: Designing Fine-Grained Systems. 2nd ed. Sebastopol, CA: O’Reilly Media.

PostgreSQL Global Development Group. 2025. „PostgreSQL Documentation.“ ბოლოს ნანახია 2026 წლის 19 იანვარს. <https://www.postgresql.org/docs>.

- Richardson, Chris. 2019. Microservices Patterns: With Examples in Java. Shelter Island, NY: Manning Publications.
- Rousselet, Rémi. 2025. „Riverpod Documentation.“ ბოლოს ნანახია 2026 წლის 21 იანვარს. <https://riverpod.dev>.
- საქართველოს ტურიზმის ეროვნული ადმინისტრაცია. 2025. „ტურიზმის სტატისტიკა.“ ბოლოს ნანახია 2026 წლის 15 იანვარს. <https://gnta.ge>.